COMPREHENSIVE PROJECT REPORT

Enterprise Resource Planning Architecture Specification

1. EXECUTIVE SUMMARY

Business Context

Authentic Lanka Exports (Pvt) Ltd is a leading agricultural manufacturing and wholesale distribution enterprise specializing in premium Sri Lankan exports. Operational expansion has introduced deep logistical and tracking complexities across multiple internal departments, including cataloging, multi-warehouse management, factory-floor processing, downstream sales, international freight shipping, accounting ledgers, and workforce payroll administration.

System Purpose

The Authentic Lanka Exports ERP is a unified, real-time Enterprise Resource Planning system engineered to replace disconnected legacy worksheets and isolated management tools. The software platform serves as a single source of truth across the enterprise, automating:

- **Product Cataloging:** Standardized multi-category item indexing utilizing a Julian date sequence naming schema.
- **Procurement (GRN):** Logging precise inbound raw farm weights, processing adjustments, and quality rejection thresholds.
- **Manufacturing Execution (MES):** Monitoring recipe formulations (BOM), wood/electricity resource utility loads, batch efficiencies, and Quality Control (QC) laboratory pipelines.
- **Logistics & Shipments:** Controlling ocean container logs, automated Security Gate Passes, and company fleet maintenance schedules.
- **Financial Ledgers:** Managing corporate Petty Cash accounts, Cheque tracking ledgers, bank balances, and a real-time Daily Profit & Loss matrix.
- **HR & Payroll:** Tracking swiped biometric attendance logs, live leave balance records, and automated monthly salary payouts.

2. TECHNICAL STACK & SYSTEM ARCHITECTURE

The core enterprise platform is structured as a modern, decoupled MERN application architecture leveraging real-time websocket events and atomic backend transaction blocks:

Frontend Architecture

Framework & Compilation: Built using React.js with Vite for high-speed module compilation and instant Hot Module Replacement (HMR).
Styling Engine: Vanilla CSS alongside Tailwind CSS utility frameworks optimized for a crisp, light, highly intuitive user layout aesthetic.
State Store: Zustand lightweight stores handle persistent user sessions, active authorization tokens, and immediate notification flags.
Server State Synchronization: React Query (TanStack Query) handles background data fetching, state mutations, server pagination caching, and proactive layout refetch cycles.
UI Components: Lucide React iconography packages tied with customized premium select inputs, badge states, paginated tables, and interactive dropdown elements.

Backend Architecture

Runtime Framework: Node.js with the Express.js micro-framework for building high-performance RESTful API endpoints.
Database Object Mapper: Mongoose ODM enforces structural document schemas over MongoDB's flexible, non-relational storage layers.
Security Authorization: JSON Web Token (JWT) architecture using standard Bearer authorization security headers.
Real-Time Synchronizations: Socket.io integration maps real-time data push conduits across active operator screens, executing immediate dashboard data adjustments when stock or shipping counts change.
Utilities: Bcryptjs handles cryptographic password salting and hashing, while logger instances (Winston and Morgan) track raw system network request logs.

3. DATABASE SCHEMA DESIGN (KEY COLLECTIONS)

The database architecture maps logical data relationships across highly indexed MongoDB document collections:

A. Core User & Employee Models

• **User:** Stores secure login criteria (email, hashed password), state flags (`isActive` , `failedLoginAttempts` , `lockedUntil`), and fine-grained authorization vectors (role designation, custom permission override arrays).
• **Employee:** Contains strict human resource parameters, linked directly to a User login profile, detailing corporate departments, designations, assigned shifts, biometric schedules, and salary structures.

B. Catalog & Inventory Models

• **Product:** Tracks item master records, category parameters, custom units of measure (UOM), wholesale baseline pricing structures, and national tax parameters.
• **Warehouse:** Manages localized storage coordinates (e.g., `MAIN`, `TRANSIT`) and real-time physical stock counts.
• **Stock & Stock Movement:** Traces stock counts across warehouses, keeping an immutable historical ledger tracking chronological movements (e.g., "Purchase Receipt", "Batch Consumption", "Sales Dispatch").

C. Purchasing & Production Models

• **Supplier & Customer:** Unified corporate registry tracking core metadata, billing structures, historical credit limits, and aging terms parameters.
• **Purchase Order (PO):** Details formal procurement contracts, raw catalog line-items, negotiated unit pricing, and lifecycle tracking states.
• **Goods Receipt Note (GRN):** Tracks exact incoming packages arriving at receiving points, recording accepted vs. rejected weights and perishability expiries.
• **Bill of Materials (BOM):** Enforces precise batch manufacturing recipes, ingredient weights, estimated processing wastage variables, and default labor/overhead financial weights.
• **Production Batch:** Tracks real-time active production floor runs, recording headcount, wood/electricity draws, output weights, batch efficiency rates, and Quality Control parameters.

D. Sales, Logistics & Finance Models

• **Sales Order (SO):** Houses customer checkout records, payment statuses, terms, and privileged sales manager credit-limit bypass overrides.
• **Invoice & Credit Note:** Manages legal business tax invoices, outward credit note adjustments, and transactional tax logs.
• **Gate Pass:** Acts as security check documents tracking vehicle details, driver verification metrics, and authorized item dispatches.
• **Fleet / Vehicle & Trip Log:** Monitors company transportation assets, log books, fuel expenditures, mileage tracking, and routine maintenance entries.
• **Petty Cash:** Records ledger adjustments for daily floor operating expenses, expense categories, and managerial validation tracking signatures.
• **Daily P&L:** Automated end-of-day compiled table computing precise net gross revenues against core operational spending metrics.

4. CORE MODULES & CALCULATION ENGINES

To eliminate computational errors, all mathematical operations are calculated server-side using atomic, precision-based functions:

A. Procurement Inspection

When raw agricultural materials arrive at receiving areas, checking inspectors submit exact parameters, and the server computes net quantities using the following programmatic formula:

```
Accepted Quantity = Received Quantity - Rejected Quantity
```

If the specific material profile features an active manufacturing conversion rule, the software runs an immediate conversion calculation to display processing yield forecasts:

$$\text{Live Yield Forecast} = \text{Accepted Quantity} \times \text{Conversion Coefficient}$$

B. Bill of Materials (BOM) Costing

When compiling or tuning a recipe sheet configuration, the core server engine dynamically calculates real-time wholesale production baseline values:

$$\text{Total Cost} = \Sigma \left[\text{Qty}_i \times \left(1 + \text{Wastage\%}_i / 100\right) \times \text{Unit Cost}_i \right] + \text{Labor Cost} + \text{Overhead Cost}$$

C. Production Batch Performance & Analytics

Upon closing a factory manufacturing run, the core tracking script computes performance metrics:

$$\text{Batch Efficiency \%} = \left(\text{Total Output Weight} / \text{Total Input Weight}\right) \times 100$$

To assist scheduling operations, the analytics environment tracks two distinct forecast calculations:

1. **Linear Regression Trends:** Fits past run index data points (x) against historical yield efficiency scores (y) utilizing standard least-squares regression modeling (`y = mx + c`) to predict future batch run yield parameters:

   $$m = \left[N\Sigma(xy) - \Sigma x \Sigma y\right] / \left[N\Sigma(x^2) - (\Sigma x)^2\right]$$
   $$c = \left[\Sigma y - m\Sigma x\right] / N$$

2. **Moving Averages:** Tracks rolling averages spanning the 5 most recently finalized production batch yields to present immediate operational benchmarks.

Additionally, resource consumption parameters are tracked using standard coefficients:

$$\text{Firewood Rate} = \text{Firewood Consumed (Kg)} / \text{Total Input Weight (Kg)}$$
$$\text{Electricity Rate} = \text{Electricity Consumed (kWh)} / \text{Total Input Weight (Kg)}$$

5. UNIQUE KEY & CODE GENERATION FORMULAS

All system documents are structurally indexed via distinct automated formatting rules to guarantee absolute unique uniqueness across transactions:

A. Products Julian Date Code

Generates an immutable unique catalog code automatically upon choosing an item category mapping rule:

```
Format: P-[CategoryShortCode]-[YearShort][JulianDayOfYear]-[SequenceNo]
```

- **P-:** Static string prefix distinguishing catalog product documents.
- **CategoryShortCode:** 3-letter capital classification value (e.g., `RAW` = Raw Material, `FNG` = Finished Goods, `MAC` = Mechanical Assets).
- **YearShort:** 2-digit token tracking the calendar year (e.g., 2026 converts to `26`).
- **JulianDayOfYear:** 3-digit padded day parameter ranging from 001 to 365/366.
- **SequenceNo:** 2-digit zero-padded day counter initializing from 01 that resets daily per product category grouping.

Example Identity Outcome: P-RAW-26155-02 identifies the 2nd distinct raw agricultural material profile created on Julian Day 155, 2026.

B. Production Batch Code

Tracks batch processing runs back to specific crop suppliers to guarantee strict upstream supply traceabilities:

```
Format: [SupplierShortCode]-ALE[YearShort][JulianDayOfYear]
```

- **SupplierShortCode:** String key identifier of the originating source farm supplier (e.g., `CHAMINDA`).
- **ALE:** Hardcoded prefix specifying the centralized factory batch execution engine.
- **YearShort & JulianDayOfYear:** Marks the exact tracking date of batch processing execution.

Example Identity Outcome: CHAMINDA-ALE26002 maps batch outputs back to farm stock delivered by Chaminda that entered factory lines on the 2nd day of year 2026.

6. CONCURRENCY, SECURITY & OPERATIONAL SAFEGUARDS

The platform embeds robust transactional and framework security architectures to prevent concurrency errors and access vulnerabilities:

A. Race-Condition Protection

To completely block sequence code naming overlap when multiple concurrent employees register items or log dispatches at the same time, the server avoids traditional separated *"read-then-write"* application query operations. Instead, it utilizes MongoDB's database-level atomic command `findOneAndUpdate` combined with the `$inc` counter operator. This approach modifies structural row increments directly within system database memory, preventing code collision risks.

B. Login Security & Locking Mechanisms

- **Brute-Force Guard:** Implements a hard cutoff threshold of 5 failed password/login attempts.
- **Account Lockout:** Upon cross-checking the 5th failed try, the script commits a `lockedUntil` timestamp parameter blocking matching login attempts for **15 minutes**.
- **Active State Checks:** Suspended or deactivated staff records flag a block instantly via the authentication routing middleware.
- **Session Life:** Uses structured JWT access tokens with absolute expiration timers; expired user sessions force automatic dashboard redirects to the login view.

C. Database Performance Controls

Daily sequence counters rely on rolling 30-day Time-To-Live (TTL) index parameters. Inactive temporal serial tracking metrics automatically purge from system index layers after 30 days of quiet state, maintaining high-speed lookups while permanently preserving transaction records.

7. DATA SEEDING & DEPLOYMENT PROCEDURES

The system deployment framework features automated initialization scripts to populate baseline records into clean target database clusters:

Seeding Workflow Sequence

- **seedDefaults.js:** Automates creation of base corporate Units of Measure (UOM), product categories, target client custom groups, the primary operational warehouse, standard Sri Lankan national tracking holidays for the year 2026, and the primary master Admin profile:
  - Username: `admin@example.com`
  - Password: `Admin123!`
- **seedPermissions.js:** Establishes complete internal system permissions structures and applies authorization mapping rules for all predefined roles (including Admin, Warehouse Manager, Accountant, HR Manager).
- **seedManufacturing.js:** Registers standard factory processing routes, sequence process templates, and yield conversion multipliers.
- **seed_exports.js:** Injects mock inventory items, verified testing suppliers, dummy customer profiles, and baseline opening stock metrics to prepare the platform for immediate validation and user acceptance testing.